# 商家后台与可观测性（v2 扩展版）

BossID 数据基础 / merchant API 与入驻 / 商家看板 / Prometheus + Grafana / ELK / 告警分级 / SLO 与 trace 驱动决策

---

gomall · 业务侧 deck

# 今日大纲

商家数据现状与 merchant 角色路线图

merchant API 草案与入驻流程

商家看板：4 象限信息架构

Jaeger：链路追踪解决的真实问题

Prometheus 业务 metrics + Grafana 大盘

ELK / EFK：日志集中化的工程化路线

静默降级 + SLO + trace 驱动决策

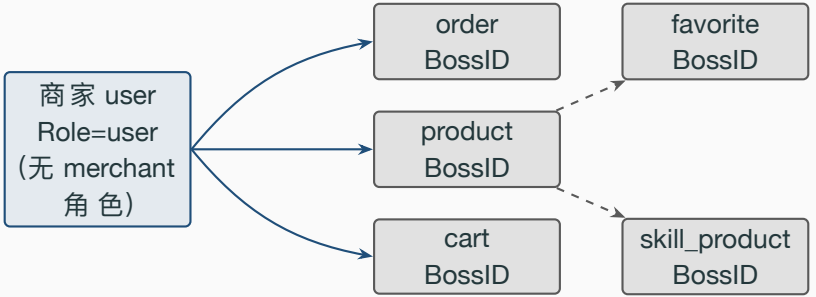
# 商家数据现状与 merchant 角色路线图

# 一个商家在 gomall 里能干什么、不能干什么

- **能干**：注册账号、上架商品、被买家下单、被买家收藏。
- **不能干**：查”我的今日营收”、查”我的待发货”、查”我的库存告警”、自助退款。
- 五张核心表里 **BossID 字段早就埋好了**：`order.BossID`、`product.BossID`、`cart.BossID`、`favorite.BossID`、`skill_product.BossID`。
- 但是 `/api/v1/merchant/*` **一个路由都没有**——数据有、查询入口缺。

*repository/db/model/order.go:12、product.go:23、cart.go:10、favorite.go:12 五处 BossID*

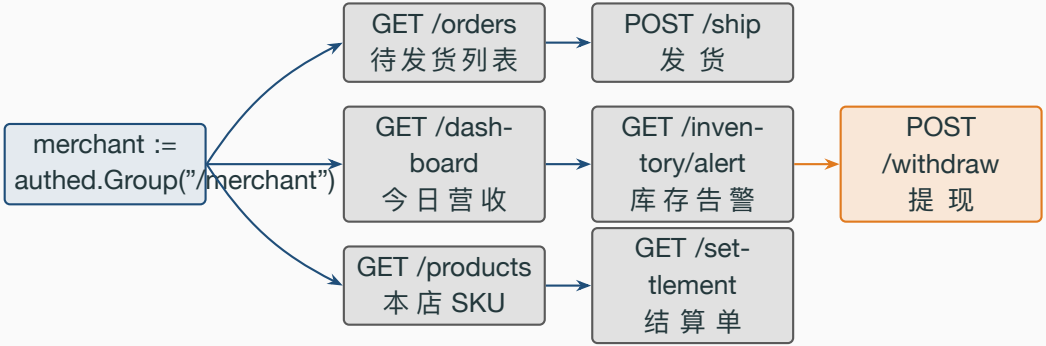
# BossID 散落在五张表里




五个外键**物理上都在了**，只是**逻辑上没有 merchant API 把它们串起来**。补一个 merchant group 不需要改 schema，只是补 5–10 个 handler。*repository/db/model/order.go:12*、*product.go:23*、*cart.go:10*、*favorite.go:12*

# merchant API 草案与入驻流程

# merchant Group 总览




7 个接口分三层：**查（dashboard / orders / products / inventory / settlement）**、**操作（ship）**、**资金（withdraw）**。资金类需要二次鉴权（短信/邮箱 OTP）。*routes/router.go:157–164*

*admin* 子组模板，*merchant* 完全复用该姿势

# 7 个接口的签名与权限矩阵

| 路径 | 方法 | 入参 | 返回 | 备注 |
|---|---|---|---|---|
| /merchant/dashboard | GET | date=YYYY-MM-DD | DashboardVO | 命 Redis 60s 缓存 |
| /merchant/orders | GET | status=wait_ship,page,size | []OrderVO | 默认 status=wait_ship |
| /merchant/orders/ship | POST | order_id, ship_no, carrier | status | 幂等 + 写 outbox |
| /merchant/products | GET | on_sale=true, page, size | []ProductVO | WHERE boss_id=? |
| /merchant/inventory/alert | GET | threshold=10 | []SkuAlert | ZRANGEBYSCORE |
| /merchant/settlement | GET | month=YYYY-MM | SettleVO | 上月汇总，T+30 |
| /merchant/withdraw | POST | amount, bank_acct | OrderNum | OTP + 风控 + 异步 |

权

**限统一**：`RequireRole("merchant","admin")` ——admin 可在出问题时**代为操作**，便于客服兜底。**数据隔离**：所有查询 SQL 末尾强制 `AND boss_id=ctx.UserId`，由 service 层注入，handler 不接受 `boss_id` 入参。*repository/db/dao/product.go:67–72 已有 WHERE id=? AND boss_id=?* 模板可复用

# merchant.Group 注册：18 行代码

Listing 1: *routes/router.go merchant.Group* 草案；与 *admin.Group* 同构

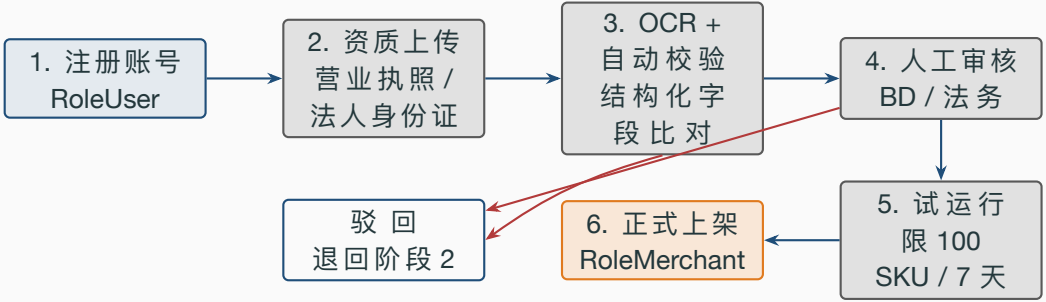
```go
// 商家后台（新增）—— 与 admin 子组同级，
// 资金类接口再叠一层 OTP，由 service 层校验
merchant := authed.Group("/merchant")
merchant.Use(middleware.RequireRole("merchant", "admin"))
{
   merchant.GET("dashboard", api.MerchantDashboardHandler())
   merchant.GET("orders", api.MerchantListOrdersHandler())
   merchant.POST("orders/ship",
      middleware.Idempotency(),
      api.MerchantShipOrderHandler())
   merchant.GET("products", api.MerchantListProductsHandler())
   merchant.GET("inventory/alert",
      api.MerchantInventoryAlertHandler())
   merchant.GET("settlement",
      api.MerchantSettlementHandler())
   merchant.POST("withdraw",
      middleware.Idempotency(),
      api.MerchantWithdrawHandler())
}
```

# merchant.Group 草案逐行讲解

- **第 3 行 Group(”/merchant”)**：与 admin 子组同级挂在 `authed` 下，自动继承 `AuthMiddleware()` ——没登录的请求根本进不到 RequireRole。
- **第 4 行 RequireRole(”merchant”,”admin”)**：变长参数复用，admin 自动有 merchant 权限，避免” 测试一个商家场景要新建一个用户” 的反复切换。
- **第 9 行 ship / 第 17 行 withdraw 加 Idempotency()**：两类**产生外部副作用**的接口（生成运单号、扣账户余额）；查询接口不需要幂等。
- **未加 SlidingWindow**：商家后台是低频管理界面，单商家 QPS 几乎不会触发限流；加了反而增加 Redis 压力。
- **未在路由层做 boss_id 校验**：boss_id 来自 `ctl.GetUserInfo(ctx).Id`，绝不从请求 body / query 读，避免越权。

*middleware/auth.go AuthMiddleware；middleware/rbac.go:48–86 RequireRole；middleware/idempotency.go*

# 从注册到正式上架：5 个阶段




整个流程**平均 3–5 个工作日**：OCR 即时返回，人工审核占大头（24–48h）；试运行用于发现资质合规但实际运营粗糙的卖家。建议新增表 *merchant_apply(user_id, license_no, status, reviewer_id, ...)*；*service/merchant_apply.go* 待建

# 阶段 2–3：资质上传与 OCR 字段校验

- **上传走 OSS**：`POST /merchant/apply/upload` 复用 `api/v1/upload.go` 的 OSS 上传链路，返回 URL。
- **OCR 拉的字段**：营业执照号、企业名称、法人姓名、注册资本、经营范围。
- **自动校验三件套**：(a) 营业执照号 18 位且校验位合法；(b) 法人姓名与身份证 OCR 一致；(c) 申请人手机号实名认证与法人姓名一致（运营商接口）。
- **三项任一不过 → status=auto_rejected**，直接返回错误码 `80001 " 您的店铺还在审核"`，但内部状态记 `auto_rejected_reason=" 法人姓名不一致"`，客服可看不能改。
- **OCR 服务挂了怎么办**：降级为” 手动录入字段 + 人工目检图片”，不阻塞流程；与 deck 10 静默降级哲学一脉相承。

*api/v1/upload.go OSS 上传链路；建议新建 service/ocr.go 抽象 OCR 供应商*

# 阶段 4–5：人工审核与试运行

- **审核界面**：admin 后台新增 `/admin/merchant/apply/list` + `/admin/merchant/apply/decide`。BD / 法务双签：BD 看品类匹配度，法务看资质合规。
- **审核 SLA**：48 小时内一审。超时进入”催办池”，自动 主管。
- **试运行限额**：通过审核后状态进入 `trial`，`RoleMerchant` 已生效，但隐性加上 (a) SKU 数 $\leq$ 100，(b) 单日订单 $\leq$ 500，(c) 不能领大额营销资源位。
- **试运行毕业条件**：7 天内 (a) 无客诉升级 (b) DSR $\geq$ 4.5 (c) 履约率 $\geq$ 95%。三项达标自动升 `normal` 状态。
- **异常退出**：试运行期 P0/P1 客诉 $\rightarrow$ 状态回退 `trial_failed`，BD 介入复盘。

建议复用 *service/admin.go:46–62 PromoteToAdmin* 模板做 *PromoteToMerchant*；试运行限额可在 *RBAC* 之后挂一个 *trial* 中间件

# 商家看板：4 象限信息架构

# 商家看板首屏的 4 个象限

**今 日 营 收**
GMV / 单数 / 客单价
order WHERE
Type=2

**待 发 货**
超 24h 红 字
order WHERE
status=wait_ship

**库 存 告 警**
< 商家自设阈值
product.Num

**客 诉 工 单**
未结案 + 今日新增
ticket（待 建）

4 个象限**对应商家一天打开后台的**

**4 个动作**：(a) 看赚了多少（左上），(b) 看要发什么（右上），(c) 看要补什么货（左下），(d) 看要处理什么投诉（右下）。**每个象限独立加载、独立缓存**，弱依赖不互相拖累。

*repository/db/model/order.go:12 BossID + Type；product.go:23 BossID + Num*

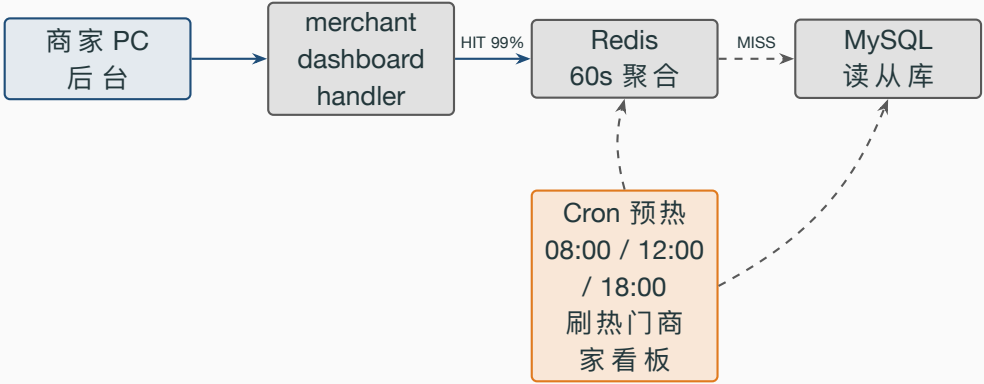
# 4 象限的数据源与计算口径

| 象限 | SQL/Redis 来源 | 计算口径 | 缓存策略 | 谁负责 |
|---|---|---|---|---|
| 今日营收 | order(BossID, Money, Type=2, created_at) | SUM(Money) GROUP BY 日 | Redis 60s | service/merchant |
| 待发货 | order(BossID, status=wait_ship) | COUNT + 列出超 24h | 列表 30s | service/merchant |
| 库存告警 | ZSET inv_alert:{boss} | ZRANGEBYSCORE 0 阈值 | 实时 | service/inventory |
| 客诉工单 | ticket(BossID, status='open') | COUNT 未结 | 缓存 60s | service/ticket（新 |

营收口径要在 deck 里讲清楚：**只算已支付（Type=2）；退款的减回去**（需要订单状态机 7 态完整落地后再加 `Refunded` 维度）。**缓存粒度按商家**：key 形如 `merchant:dashboard:{boss_id}:{date}`，不同商家不会互相挤压。

*repository/db/model/order.go Type=2 已支付；订单状态机 7 态见 deck 09 v2*

# 看板架构：读分离 + 三层缓存 + 异步预热




- **读分离**：所有查询打**从库**，避免占用下单链路的主库连接池。
- **60s 缓存**：商家可接受 1 分钟的数据延迟（不是炒股盘）；同时把 DB 压力砍到 1/60。
- **异步预热**：Cron 在三个高峰前把头部商家的看板提前算好，确保 p99 不抖。

# 今天只有两层墙，明天要加中间一层

| 角色 | 路径 | 现状 | 备注 |
|---|---|---|---|
| user | `/api/v1/*` | ✓ | 默认放行 |
| merchant | `/api/v1/merchant/*` | **缺** | 中间一层待建 |
| admin | `/api/v1/admin/*` | 部分 | 已有 3 个 handler |

第三层 admin 已落地

`(RequireRole("admin"))`；**缺的是中间一层 merchant**。*routes/router.go:148–155 admin 子组；middleware/rbac.go:48–86 RequireRole*

# merchant 角色落地需要改的四处

1. `repository/db/model/user.go:27--32`：新增常量 `RoleMerchant = "merchant"`。Role 字段已有，无需 migration。
2. `middleware/rbac.go`：`RequireRole` 已是变长参数 `(allowed ...string)`，调用方传 `"merchant","admin"` 即可。
3. `routes/router.go`：新增 `merchant := authed.Group("/merchant") + merchant.Use(middleware.RequireRole("merchant","admin"))`。
4. `service/admin.go`：补一个 `PromoteToMerchant`，参考 `PromoteToAdmin:46--62` 直接复刻；记得调 `InvalidateRoleCache`。

四处改动各 5–15 行，不需要任何 DB migration。整套路线图**两个 PR 就能合**。

*routes/router.go:148–155* 现成模板；*service/admin.go:46–62 promote* 逻辑可复用

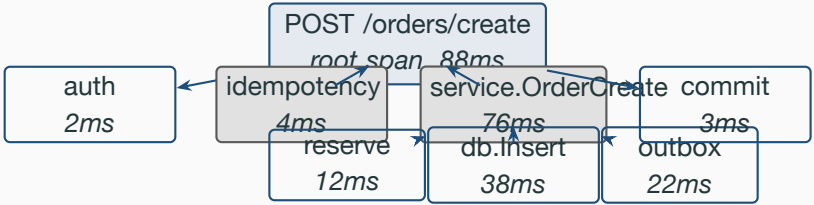
# RequireRole + lookupRole：30 秒缓存的取舍

Listing 2: *middleware/rbac.go:22–45 + :89–91*

```go
var (
   roleCache sync.Map
   roleCacheTTL = 30 * time.Second
)
func lookupRole(ctx context.Context, userId uint) (string, error) {
   if v, ok := roleCache.Load(userId); ok {
      e := v.(roleCacheEntry)
      if time.Now().Before(e.expires) { return e.role, nil }
   }
   u, err := dao.NewUserDao(ctx).GetUserById(userId)
   if err != nil { return "", err }
   role := u.Role
   if role == "" { role = "user" }
   roleCache.Store(userId, roleCacheEntry{role: role,
      expires: time.Now().Add(roleCacheTTL)})
   return role, nil
}
func InvalidateRoleCache(id uint) { roleCache.Delete(id) }
```

# Jaeger：链路追踪解决的真实问题

# 一次下单的 trace span 树




一棵树就把” 慢在哪一段” 答清楚。**root 88ms 中 db.Insert 占 38ms**，是该接口下一步优化的着力点。

*service/order.go OrderCreate 主路径；track.WithSpan 在子函数里开 child span*

# Jaeger middleware：traceId 透传

Listing 3: *middleware/track.go:11–29*

```go
11 func Jaeger() gin.HandlerFunc {
12    return func(c *gin.Context) {
13       traceId := c.GetHeader("uber-trace-id")
14       var span opentracing.Span
15       if traceId != "" {
16          var err error
17          span, err = track.GetParentSpan(
18             c.FullPath(), traceId, c.Request.Header)
19          if err != nil { return }
20       } else {
21          span = track.StartSpan(
22             opentracing.GlobalTracer(), c.FullPath())
23       }
24       defer span.Finish()
25       c.Set(consts.SpanCTX, opentracing.ContextWithSpan(c, span))
26       c.Next()
27    }
28 }
```

# Jaeger middleware 逐行讲解

- **第 3 行**：读 `uber-trace-id` header；格式 `<traceId>:<spanId>:<parentId>:<flags>`。
- **第 5–9 行**：有 traceId → 当前请求是别人调过来的，开 **child span**，root 留在上游。
- **第 10–13 行**：没 traceId → 当前请求是入口，自己开 root span。
- **第 14 行**：`defer span.Finish()` 必须有，否则 Jaeger 端会看到”未结束”灰色 trace。
- **第 15 行**：把 span 写回 `c.Context`，下游通过 `ctl.GetSpanContext` 开 child span ——**这一步就是”链路”的串接**。

*pkg/utils/track/track.go:61–78 GetParentSpan Extract*

# Jaeger 的开销：实测低于 5%

| 场景 | RPS | p95 |
|---|---|---|
| `/ping` 裸基线（含 ratelimit/cors/Jaeger 全中间件） | 64,254 | 3.51ms |
| `/product/show` 真业务（MySQL PK + 全中间件） | 62,226 | 3.01ms |

- `/ping` 已经过了 `Jaeger()`（router.go:23 注册），不是” 无 trace 裸 RPS”。
- 商品详情 RPS 仅比 ping 低 **3.2%**，证明全链路 trace 在 const sampler 全量采样下，单机损耗 $<$ 5%。
- 真把损耗摁住，还能从 `const` 切到 `probabilistic` 抽样（默认 0.001%），CPU 占用再降一个数量级。

*stressTest/REPORT.md 链路压测表第 2、3 行；pkg/utils/track/track.go:15–27 GetDefaultConfig const sampler*

# Jaeger 与 Skywalking 同时存在的取舍

| 维度 | Jaeger（OpenTracing） | Skywalking |
|---|---|---|
| 插桩方式 | 业务代码显式开 span | agent 自动织入 |
| 学习曲线 | 中（要懂 carrier/inject） | 低（接入即用） |
| 定制空间 | 高（可任意标 tag） | 中（agent 决定） |
| UI / 拓扑 | 单服务 trace 视角 | 全链路服务拓扑图 |
| gomall 用法 | 已落地，主线 | 备选，docker 起着 |

**业务选型逻辑**：单服务调

试用 Jaeger 够细；多服务上线后看拓扑用 Skywalking。两者不冲突，可以并存——当前 gomall 把两套基础设施都备齐，主推 Jaeger，留 Skywalking 做 SRE 大盘。*cmd/main.go:44*

注释掉的 `//track.InitJaeger()` 留了切换开关

# Prometheus 业务 metrics + Grafana 大盘

# 要给业务接的 4 类 metrics

| metric | 类型 | 标签 | 用途 | 来源代码 |
|---|---|---|---|---|
| order_created_total | Counter | status, boss_id_bucket | 单数 / 转化 | service/order.go OrderCreate |
| order_create_duration_ms | Histogram | status | p50/p99 | middleware/timing.go（新建） |
| paydown_total | Counter | channel, status | 渠道占比 | service/order_payment.go |
| ratelimit_hit_total | Counter | scope | 限流命中率 | middleware/ratelimit.go SlidingW |
| circuit_state | Gauge | route | 熔断器实时态 | middleware/circuitbreaker.go stat |
| idempotency_hit_total | Counter | route | 60002 命中 | middleware/idempotency.go |

标签设计原则：**枚举值数量有限**。`boss_id` 不能直接当标签（百万级 cardinality 会把 Prometheus 内存打爆），改成 `boss_id_bucket`（按 GMV 等级分 5 档）。

*middleware/ratelimit.go SlidingWindow / middleware/circuitbreaker.go state*；指标可在两个中间件里挂

# 接入 Prometheus：30 行起步

Listing 4: *pkg/utils/metrics/metrics.go* 草案；与 *middleware/timing.go* 配合

```go
package metrics

import (
   "github.com/prometheus/client_golang/prometheus"
   "github.com/prometheus/client_golang/prometheus/promauto"
)

var (
   OrderCreatedTotal = promauto.NewCounterVec(
      prometheus.CounterOpts{
         Name: "gomall_order_created_total",
         Help: "订单创建累计计数",
      }, []string{"status", "boss_bucket"})

   OrderDurMs = promauto.NewHistogramVec(
      prometheus.HistogramOpts{
         Name: "gomall_order_create_duration_ms",
         Help: "下单耗时（毫秒）",
         Buckets: []float64{5, 10, 25, 50, 100, 250, 500, 1000}
```

# Prometheus 接入逐行讲解

- **promauto 而不是手 Register**：自动注册到默认 Registry，避免漏注册导致 metric 丢失。
- **Counter vs Histogram vs Gauge**：单调递增用 Counter；分位用 Histogram（buckets 是预定义的耗时阈值，决定 p99 精度）；当前态用 Gauge。
- **Histogram buckets 选择**：从 5ms 到 1000ms 八档，参考 `/orders/create` 实测 p95 ≈ 2.3ms（见 deck 10 压测表），5ms 起步刚好覆盖 p95 + p99。
- **标签穷举**：`status` 取 `success / fail / limited / cb_open / idempotent_hit`，5 个值；`boss_bucket` 按 GMV 分 `S / A / B / C / D`，5 个值。两个标签 5*5=25 个时间序列，可控。
- **怎么暴露**：在 router 加 `r.GET("/metrics", gin.WrapH(promhttp.Handler()))`，Prometheus 默认 15s 抓一次。

*Prometheus 官方建议 Counter / Histogram / Gauge 三类用法；stressTest/REPORT.md 第 1–3 行实测 p95 数据*

# Grafana 看板：5 个核心面板

| 面板 | 数据源 | 阈值（黄 / 红） | 告警接收人 |
|---|---|---|---|
| 订单总量趋势 | sum(rate(order_created_total[5m])) | — / — | 不告警 |
| 下单 p99 延迟 | histogram_quantile(0.99, ...) | 200ms / 500ms | SRE |
| 下单错误率 | rate(...status=”fail”...) / rate(...) | 1% / 5% | SRE + 业务 |
| 限流命中率 | rate(ratelimit_hit_total[1m]) / RPS | 0.5% / 2% | SRE |
| 熔断器开启次数 | changes(circuit_state==1[5m]) | 1 次 / 3 次 | SRE + 商家域负责人 |

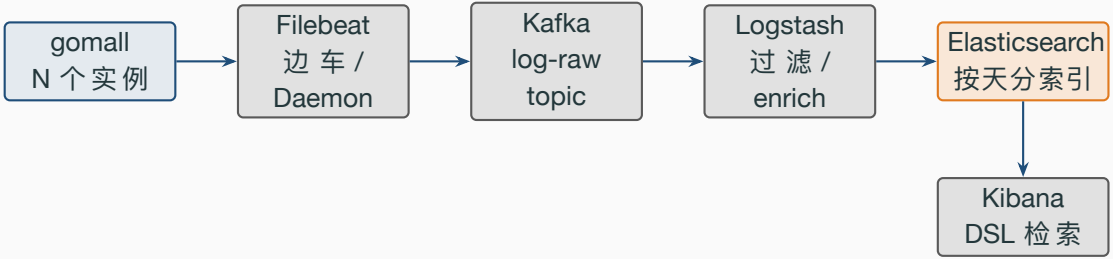
**阈值不是拍**

**脑袋**：基线数据来自压测——ping 64K RPS / p95 3.5ms 是**资源上限**，下单 50K RPS / p95 2.3ms 是**业务上限**。生产把警戒线放在压测 p95 的 5–10 倍，留出余量。

*stressTest/REPORT.md* 链路压测表第 *1*、*4* 行 *p95* 数据；阈值倍率参考通用 *SRE* 实践

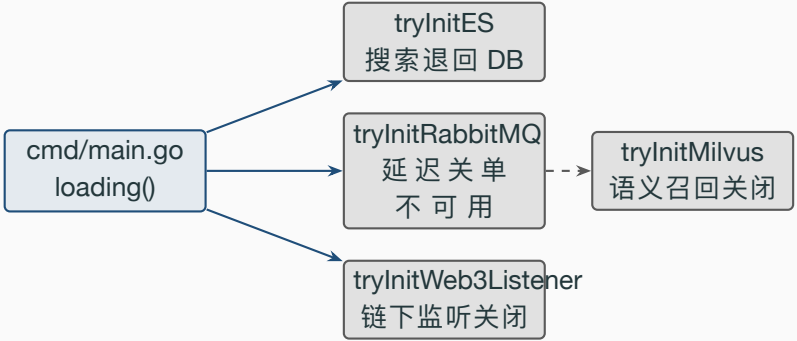
# ELK / EFK：日志集中化的工程化路线

# 规模化版本：Filebeat → Kafka → Logstash → ES




- **为什么中间加 Kafka**：Filebeat 直连 ES，ES 抖动一次全站日志丢；Kafka 做 buffer，ES 可用性事故不会丢日志。
- **Logstash 做什么**：把 `traceId` 提取成顶层字段、`userId` 脱敏（保留前 3 + 后 3 位）、`level=error` 的同时复制到 `alert` 索引。
- **按天分索引**：`gomall-app-2026.05.16`，便于按时间清理（保留 30 天，冷归档到 OSS）。

# gomall 启动期的四个 tryInit




- 四个外部依赖任意一个挂了，HTTP 主路径**照常起**：登录、浏览、下单、支付都不受影响。
- 用户最多失去某一项**增量能力**：搜不到语义结果、订单不自动延迟关闭，但不会”白屏 500”。



*cmd/main.go:37–42* 调用；*:54–97* 四个 *tryInit* 实现

# tryInitES：8 行 recover 套路

Listing 5: *cmd/main.go:64–73*

```go
// tryInitES ES 不可用时静默跳过，搜索接口退回 DB 路径
func tryInitES(ctx context.Context) {
    defer func() {
        if r := recover(); r != nil {
            util.LogrusObj.Warnf("ES 初始化失败，搜索退化：%v", r)
        }
    }()
    es.InitEs()
    initialize.InitSearch(ctx)
}
```

# tryInitES 逐行讲解：为什么不传 error

- **第 3–8 行 defer + recover**：底层 `es.InitEs()` 是**老代码**，连不上 ES 直接 `panic`。改 `InitEs` 签名风险大（连锁影响 search service），所以**在外层包一个 recover**。
- **为什么不返回 error 让 main 决定**：main 拿到 error 也只能写日志接着启——业务上"ES 挂" 和" 启动失败" 是**两件事**，前者不应阻塞 HTTP 启动。
- **为什么用 Warnf 不是 Errorf**：error 级别会触发告警；这里是" 已知降级路径"，不是异常。给 SRE 留 Warn 让他主动看，不要凌晨 3 点叫人起床。
- **相同套路用了 4 次**：`tryInitRabbitMQ`、`tryInitES`、`tryInitWeb3Listener`、`tryInitMilvus` ——同一种姿势，业务上是**对" 不可控外部依赖" 的统一回应**。

*cmd/main.go:54–97 四个 trylnit 同构*

# 业务码：现有 5 段 + 待加 80xxx 商家段

| 段 | 含义 | 现有 / 新增码 | 客服话术示例 |
|---|---|---|---|
| 20xxx | 业务通用 | ERROR=20001 | ” 请稍后再试” |
| 30xxx | 鉴权 | ErrorAuth* | ” 登录已过期” |
| 50xxx | 第三方 | ErrorOss=50001 | ” 上传失败” |
| 60xxx | 并发 / 幂等 | 60002 命中 | 静默（用户无感） |
| 70xxx | 流量治理 | 70001 限流 / 70002 熔断 | ” 正在排队 / 当前繁忙” |
| 80001 | 店铺未审核 | 新增 | ” 您的店铺还在审核，1–3 工作日” |
| 80002 | 操作非本店 SKU | 新增 | ” 该商品不属于您的店铺” |
| 80003 | 提现额度不足 | 新增 | ” 本月额度已满，下月 1 号刷新” |
| 80004 | 发货超时改判即将触发 | 新增 | ” 您有 N 单超 72h 未发货” |
| 80005 | 库存低于阈值 | 新增 | ”SKU XXX 库存 < 阈值，请补货” |

编码原则

同 deck 10：**HTTP 200 + status 字段**。商家段隔离避免与 admin / C 端文案串号。

*pkg/e/code.go* 现有 5 段；*docs/slides/10-business-traffic-governance.tex* 话术映射模板

# 静默降级 + SLO + trace 驱动决策

# 静默降级的 5 条铁律（提炼自 cmd/main.go 现状）

1. **依赖必须是”增量能力”**：搜索语义召回、订单延迟关闭、链下事件、消息异步——这些都是”锦上添花”。下单 / 支付 / 鉴权**不能进降级名单**。
2. **兜底路径必须存在**：ES 挂 → `InitSearch` 不动，搜索退回 DB `LIKE %keyword%`；Milvus 挂 → `semantic-search` 接口短路返回 `{items: []}`。
3. **recover 必须配 Warn 日志 + traceId**：不能 silent。”sliently swallow + 0 日志” 是事故制造机。
4. **recover 范围最小化**：只包 `InitX`，不包 `r.Run`。HTTP 主路径要 fail fast。
5. **对外可见性**：商家投诉” 搜不到我新上的 SKU” 时，SRE 一搜 Warn 日志 `"ES 初始化失败"` 就知道是哪种降级状态。

*cmd/main.go:54–97* 四个 *tryInit* 的同构姿势；*repository/es/* 退化路径

# 静默降级的请求期版本：handler 内的弱依赖兜底

Listing 6: *service/product_search.go* 草案；与 *cmd/main.go:64–73* 同构

```go
// SearchProducts 弱依赖 ES，主路径返回 DB 结果，
// ES 命中则用 ES 的更优排序覆盖
func (s *ProductSrv) SearchProducts(
    ctx context.Context, kw string) ([]*model.Product, error) {

    // 1. 主路径：DB LIKE，必须返回
    dbResult, err := s.dao.SearchByKeyword(ctx, kw)
    if err != nil { return nil, err }

    // 2. 锦上添花：ES 提供更好的排序
    if !es.Available() {
        log.WithCtx(ctx).Warnf("ES 不可用，搜索降级到 DB")
        return dbResult, nil // 主路径已经够用
    }
    esResult, err := s.esRepo.SearchProducts(ctx, kw)
    if err != nil {
        log.WithCtx(ctx).Warnf("ES 查询出错，降级：%v", err)
        return dbResult, nil
    }
```

# 请求期降级套路逐行讲解

- **第 5–6 行**：DB 是**底盘**，先拿到结果。ES 用还是不用，是优化问题，不是功能问题。
- **第 9–10 行 es.Available()**：bool 缓存上一次健康检查结果（每 5s 更新），避免每次请求都打 ES。
- **第 11 行 log.WithCtx(ctx).Warnf**：traceId 自动注入，客服报” 搜不到” → Kibana 搜 traceId 一眼看清是降级了。
- **第 14–17 行**：ES 报错时**不 return error**，因为 DB 已经给了答案，用户根本不需要知道 ES 有事。
- **第 18 行 mergeResults**：把 ES 的语义排序 + DB 的 keyword 全量**融合**，ES 命中段排前，未命中段按 DB 顺序排后——见 deck 21 v2 hybrid 排序。

*cmd/main.go:64–73 启动期同构；deck 21 v2 hybrid 排序公式*

# 99.9 / 99.95 / 99.99 各档预算

| SLO | 月不可用预算 | 一次事故上限 | 适用场景 | 成本 | gomall 当前 |
|---|---|---|---|---|---|
| 99.9% | 43m 12s | 30 分钟 | 商家后台 / 后台运营 | 单机即可 | 现状 |
| 99.95% | 21m 36s | 10 分钟 | 商品浏览 / 搜索 | 主从 + Redis | 现状 |
| 99.99% | 4m 19s | 2 分钟 | 下单 / 支付 | 多 AZ + 蓝绿 | 目标 |
| 99.999% | 26s | — | 不做（性价比极低） | — | — |

- **预算思想**：SRE Book 经典——” 可靠性不是越高越好，多出来的可靠性预算应当转成产品迭代速度”。
- **gomall 优先级**：下单 / 支付保 99.99%，商家后台保 99.9% 即可（商家可以等 15 分钟，但用户不能）。
- **预算耗尽怎么办**：当月剩余预算 < 50% 时，**冻结非紧急上线**。

*Google SRE Book Ch.4 SLO*；阈值参考通用电商二线对标

# P0–P3 告警分级与响应 SLA

| 级别 | 触发条件示例 | 响应 SLA | 通知方式 | on-call 行为 |
|---|---|---|---|---|
| P0 | 下单错误率 > 5%、支付完全不可用 | 5 分钟 | 电话 + IM 全员 | 立即上线，主管同时介入 |
| P1 | 下单 p99 > 1s 持续 10 分钟、ES 全挂 | 15 分钟 | 电话 + IM | 主 on-call 上线 |
| P2 | 限流命中率 > 2%、熔断器 5 分钟 3 次 | 30 分钟 | IM 通知 | 工作时间内排查 |
| P3 | Warn 日志聚集、商家后台慢 | 4 小时 | IM 静默 | 次日复盘 |

- **分级原则**：不是” 看告警严重程度”，是” 看用户感知 + 业务收入”。下单挂了影响 GMV，P0；商家后台慢 P3 给 BD 解释一下就行。
- **on-call 轮值**：每周一轮换，工作日 + 周末分两人；P0 双 on-call 自动同时呼叫。
- **post-mortem**：所有 P0 / P1 事故 7 天内必须出 blameless 复盘，沉淀到 wiki。

告警阈值与本 *deck ”Grafana* 看板：*5* 个核心面板” 阈值表对齐

# SLO 燃尽预算（Error Budget Burn Rate）

- **Burn rate** = 当前错误率 ÷ SLO 允许的错误率。
- 下单 `SLO=99.99%` → 允许错误率 0.01%。当前 5 分钟内实测错误率 0.05% → burn rate = 5。
- **阈值告警**：
  - burn rate > 14.4：1 小时烧完月预算 → P0
  - burn rate > 6：6 小时烧完 → P1
  - burn rate > 3：1 天烧完 → P2
  - burn rate > 1：1 个月烧完（即只是触线）→ P3
- **为什么不用静态阈值**：静态阈值在高峰期容易误报、在低谷期容易漏报；burn rate 是**相对预算的速率**，跨流量级别都准。

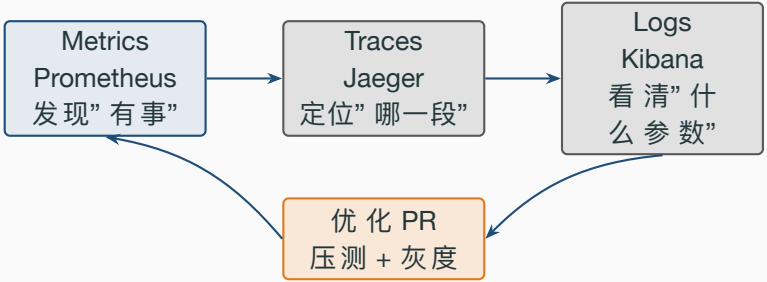
*Google SRE Workbook Ch.5 Alerting on SLOs 烧穿率告警*

# 真实案例：从 trace 找到订单列表深分页问题

- **症状**：商家投诉”翻到第 100 页特别慢”。客服无 traceId 可贴，先用 Kibana 按 `route: "/orders/old/list"` 拉日志，发现 90% 请求耗时 > 5s。
- **用 trace 定位**：在 Jaeger 按 `op=GET /orders/old/list` 拉 p99 trace 一条，看到 **db.Query span 占 root 的 99%**，扫了 4M 行。
- **优化方案**：游标分页 + Redis 5min 首页缓存（见 PR #38）。
- **产出效果**：压测对照——旧版 **8.3 RPS / p95 15.95s**，优化版 **58,406 RPS / p95 5ms**。吞吐 7000x，延迟 3200x。
- **决策闭环**：trace 看出瓶颈 → PR 优化 → 压测对比 → 灰度 → Grafana 监控 p99 持续观察 1 周。

*stressTest/REPORT.md 第 8、3 行；PR #38 游标分页*

# trace + metrics + log 三件套的协同闭环




三件套缺一不可：单看 metrics 知道有事但不知道在哪；单看 trace 看个体但不知道总体；单看 log 看到细节但找不到趋势。**三者联查 + PR 闭环 + 灰度复盘 + 再监控**，是可观测性的真正闭环。

> 可观测性不是”把工具都装上”，而是**形成”发现 → 定位 → 优化 → 度量”的闭环**。gomall 当前到了”工具齐”的阶段，下一步是”流程齐”。

# 三类人共用一套底层信号

| 角色 | 关心 | 看什么 |
| --- | --- | --- |
| 商家 | 卖了多少、待发几单 | 商家后台（待建） |
| SRE | p99、错误率、依赖健康度 | Jaeger / Skywalking |
| 客服 | 单个 traceId 怎么挂的 | 日志 + trace 联查 |

三个视角共用**同一套信号**

（JSON 日志 + trace span 树），差别只在聚合维度。gomall 现状：信号已采集，**聚合视图还没搭**。下一步两个 PR：merchant 角色 + 看板 API；再下一个迭代：traceId 入日志 + Filebeat 接入。

> 可观测性是为了让**业务方独立解释自己的现状**——商家解释自己卖得好不好，SRE 解释系统稳不稳，客服解释这一单为什么挂。

# 面试 Q&A（1/2）：架构与权限

**Q1**：BossID 都在了，为什么不直接给商家开 admin?

A：admin 看全站数据，商家只能看自店；权限边界不能用” 先开放再监控” 做。merchant 角色 + `RequireRole("merchant")` 才是正确解。*[middleware/rbac.go:48–86]*

**Q2**：merchant.Group 为什么不在路由层校验 boss_id?

A：boss_id 从 `ctl.GetUserInfo(ctx).Id` 取，绝不进 body/query。handler 入参故意不暴露该字段——” 入参里没有 = 没法越权” 是设计层面的防御。*[pkg/utils/ctl/ctl.go]*

**Q3**：roleCache 为什么 30s 不是 5min?

A：商家被提权后等 5min 才生效，谈判桌上没法解释。30s 是” 权限漂移可接受 + 回源 DB 砍 30 倍” 的平衡点；提权时 `InvalidateRoleCache` 让**当事人立即生效**。*[middleware/rbac.go:22–45]*

**Q4**：商家入驻为什么要先 OCR 再人工审核?

A：OCR 即时返回，过滤掉资质号不合规、姓名不一致的 80% 提交，让人工只看真案子。审核 SLA 48h，人工资源精确投放。*[service/ocr.go 待建]*

**Q5**：商家看板 4 角限为什么独立加载?

# 面试 Q&A（2/2）：可观测性与降级

**Q7**：Jaeger 和 Skywalking 同时存在，是不是重复建设？

A：单服务调试 Jaeger 细，多服务拓扑 Skywalking 直观。两者数据模型相同（trace+span），并存不冲突。gomall 主推 Jaeger，留 Skywalking 做 SRE 大盘。*[cmd/main.go:7]*

**Q8**：Prometheus 的 `boss_id` 标签为什么不能直接用？

A：Prometheus 标签 cardinality 必须有限。百万级 boss_id 会把内存撑爆。改成 `boss_bucket`（按 GMV 分 5 档），时间序列数 = 标签笛卡尔积，可控。*[pkg/utils/metrics/metrics.go 草案]*

**Q9**：Filebeat 直连 ES 不行吗，为什么还要 Kafka？

A：Filebeat 直连 → ES 抖动一次全站日志丢。Kafka 做 buffer，ES 故障期间日志先进 Kafka，恢复后由 Logstash 重放。可用性 ≫ 简洁性。*[docker-compose.yml:44–99]*

**Q10**：tryInitES 为什么用 recover 不传 error？

A：底层 `es.InitEs()` 老代码 panic 风格，改签名牵连大。外层 recover + Warnf 是**对"启动期不可控依赖"的统一回应**，与请求期熔断（70002）互补。*[cmd/main.go:64–73]*

# 代码位置一览

**BossID 散落字段** *model/order.go:12, product.go:23, cart.go:10, favorite.go:12*
**RBAC 中间件** *middleware/rbac.go:22–45, :48–86, :89–91*
**Jaeger middleware** *middleware/track.go:11–29*
**Trace utility** *pkg/utils/track/track.go:15–78*
**Skywalking agent** *cmd/main.go:7 import* 副作用
**logrus 初始化** *pkg/utils/log/logger.go:14–41*
**静默降级（启动期）** *cmd/main.go:54–97* 四个 *tryInitX* 同构
**静默降级（请求期）** *service/product_search.go* 草案；*deck 21 v2 hybrid*
**merchant.Group 草案** *routes/router.go merchant := authed.Group("/merchant")*（本 *deck p.10*）
**merchant API 7 接口** *api/v1/merchant_*.go* 待建 *7* 个 *handler*
**商家入驻申请** *model merchant_apply (*待建*)*；*service/merchant_apply.go*
**Prometheus metrics** *pkg/utils/metrics/metrics.go* 草案；*middleware/timing.go*
**ELK / EFK 落地** *docker-compose.yml:44–99 ES+Kibana*；*Filebeat/Kafka/Logstash* 路线图
**admin promote 模板** *service/admin.go:46–62 PromoteToAdmin*
**admin 路由组** *routes/router.go:157–164 admin* 子组三接口
**CircuitBreaker 配置** *routes/router.go:122–130 paydown*
**压测锚点** *stressTest/REPORT.md*：*ping 64,254 / product 62,226 /* 订单优化 *58,406 vs* 旧 *8.3 RPS*

配套：18-merchant-business.tex（商家旅程）/ 10-business-traffic-governance.tex（流量治理）/ 09 v2 订单状态机 / 13 列表性能 / 21 v2 AI 检索